

LAPORAN PRAKTIKUM BIG DATA

Praktikum 1



OLEH :

JARDEL POLIVIERA

2411533012

DOSEN PENGAMPU:

AFDHAL DINILHAK. M.Kom

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG 2026

Link repo :

https://github.com/JardelPoliviera/BigData26_B_2411533012_JardelPoliviera

A. PENDAHULUAN

Analisis mobilitas perkotaan modern menuntut pemahaman yang mendalam mengenai pola pergerakan masyarakat setiap harinya. Salah satu indikator terbaik untuk mengukur aktivitas ini adalah data operasional taksi kuning yang merekam jutaan perjalanan penumpang secara rinci. Data rekaman perjalanan ini tidak sekadar berisi angka, melainkan menyimpan informasi berharga terkait jam sibuk, jarak tempuh, hingga fluktuasi permintaan layanan transportasi komersial di suatu kota.

Studi kasus ini berfokus pada perbandingan komprehensif antara dataset operasional taksi pada bulan Januari dan Februari pada tahun yang sama. Tujuan utama dari analisis komparatif ini adalah untuk mengidentifikasi apakah terdapat pergeseran tren perilaku komuter atau justru menunjukkan pola yang konsisten dari waktu ke waktu. Melalui serangkaian proses ekstraksi, pembersihan data, hingga pemetaan visual menggunakan bahasa pemrograman Python, tahapan analisis ini dirancang untuk menyingkap wawasan penting di balik tingginya volume transaksi harian..

B. TUJUAN

1. Mahasiswa dapat menyiapkan notebook Colab yang siap pakai dan mendokumentasikan spesifikasi lingkungan kerjanya (Python version, library version, RAM, disk).
2. Mahasiswa dapat mengunduh dataset publik secara terprogram dan memeriksa integritas awalnya (ukuran file, jumlah baris, skema kolom).
3. Mahasiswa dapat mengukur pemakaian memori sebuah DataFrame dan menurunkannya melalui pemilihan kolom serta optimasi tipe data.
4. Mahasiswa dapat memproses file yang lebih besar dari RAM yang tersedia menggunakan teknik chunking
5. Mahasiswa dapat menghasilkan tabel agregasi (per jam, per metode pembayaran) dan menyimpannya dalam format CSV serta Parquet.
6. Mahasiswa dapat menjalankan kueri agregasi menggunakan PySpark local mode dan menjelaskan perbedaan konseptualnya dengan pandas.

C. LANGKAH PRAKTIKUM

1. Verifikasi Lingkungan Kerja

```
import sys, platform, multiprocessing

print("Python :", sys.version.split()[0])
print("OS      :", platform.platform())
print("CPU      :", multiprocessing.cpu_count())

for nama in ["pandas", "numpy", "pyarrow", "matplotlib"]:
    try:
        mod = __import__(nama)
        print(f"{nama:11s}: {mod.__version__}")
    except ImportError:
        print(f"{nama:11s}: BELUM TERPASANG")

... Python : 3.13.15
    OS      : Linux-6.6.122+-x86_64-with-glibc2.39
    CPU      : 2
    pandas   : 2.2.3
    numpy    : 2.1.3
    pyarrow  : 23.0.1
    matplotlib : 3.10.0

!free -h      # kapasitas dan sisa RAM
!df -h /content #kapasitas dan sisa disk runtime
!java -version #dibutuhkan Pyspark pada langkah k-8

total        used        free      shared  buff/cache   available
Mem:         12Gi       1.8Gi       7.8Gi       2.2Mi       3.3Gi       10Gi
Swap:          0B           0B           0B
Filesystem    Size  Used Avail Use% Mounted on
overlay       108G   21G   88G  19% /
[0.001s][warning][os,container] Cgroup memory controller path at '/sys/fs/cgroup' seems to have moved to '/../jupyter-children', detected limits won't be accurate
[0.001s][warning][os,container] Cgroup cpu controller path at '/sys/fs/cgroup' seems to have moved to '/../jupyter-children', detected limits won't be accurate
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-24.04-Ubuntu, mixed mode, sharing)
```

Kode ini mencetak spesifikasi lingkungan yang digunakan. Output menunjukkan bahwa Python berjalan pada versi 3.13.15 dan menggunakan 2 core CPU. Library pandas, numpy, pyarrow, dan matplotlib sudah berhasil terpasang beserta versi masing-masing.

Melalui perintah shell (!free, !df, !java), output menampilkan total RAM sebesar ~12 GB dengan sisa ~11 GB, kapasitas penyimpanan tersisa sekitar 88 GB, dan Java versi 21.0.12 telah siap (dibutuhkan untuk menjalankan PySpark di langkah selanjutnya).

2. Menghubungkan Google Drive

```
from google.colab import drive
try:
    drive.mount("/content/drive", force_remount=True)
except Exception as e:
    print("Proses mount menemui kendala:", e)

import os
DIR_KERJA = "/content/data"
DIR_SIMPAN = "/content/drive/MyDrive/Colap Nootbook/Big Data/Praktikum1"

os.makedirs(DIR_KERJA, exist_ok=True)
os.makedirs(DIR_SIMPAN, exist_ok=True)
print(os.listdir(DIR_SIMPAN))

Mounted at /content/drive
['agregasi_per_jam.csv', 'agregasi_per_jam.parquet', 'pengukuran_kinerja.csv', 'distribusi_per_jam.png']
```

Menunjukkan pesan "Mounted at /content/drive", yang menandakan Google Drive berhasil dihubungkan. Output berikutnya berupa daftar file ['agregasi_per_jam.csv', ...] yang menunjukkan bahwa direktori kerja telah berhasil dibuat dan menampilkan file-file yang sudah ada di dalam folder tersebut sebelumnya.

3. Menyiapkan Alat Ukur

```
import time, os, psutil

proses = psutil.Process(os.getpid())
catatan = []

def rss_mb():
    return proses.memory_info().rss / 1024**2

def ukur(label, fungsi):
    """Menjalankan fungsi tanpa argumen, mencatat durasi dan pertumbuhan memori."""
    m0, t0 = rss_mb(), time.perf_counter()
    hasil = fungsi()
    detik = time.perf_counter() - t0
    delta = rss_mb() - m0
    catatan.append({"langkah": label, "detik": round(detik, 2), "delta_rss_mb": round(delta, 1)})
    print(f"[{label}] {detik:.2f} s | RSS delta: {delta:+.1f} MB")
    return hasil
```

Tidak ada output visual yang dihasilkan pada sel ini karena fungsinya hanya untuk mendefinisikan sebuah fungsi Python (ukur dan rss_mb) yang akan digunakan untuk mengukur waktu eksekusi (detik) dan penambahan memori (MB) pada sel-sel berikutnya.

4. Mengunduh Dataset dan Inspeksi Awal

```
import urllib.request
URL = ("https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2023-01.parquet")
PATH = os.path.join(DIR_KERJA, "yellow_tripdata_2023-01.parquet")

if not os.path.exists(PATH):
    ukur("unduh dataset", lambda: urllib.request.urlretrieve(URL, PATH))

print("Ukuran file:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")

import pyarrow.parquet as pq
meta = pq.ParquetFile(PATH).metadata

print("Jumlah baris :", f"{meta.num_rows:,}")
print("Jumlah kolom :", meta.num_columns)
print("Row group      :", meta.num_row_groups)
print("Nama kolom     :", pq.ParquetFile(PATH).schema.names)
```

[unduh dataset] 0.35 s | RSS delta: +1.8 MB
Ukuran file: 45.5 MB
Jumlah baris : 3,066,766
Jumlah kolom : 19
Row group : 1
Nama kolom : ['VendorID', 'tpep_pickup_datetime', 'tpep_dropoff_datetime', 'passenger_count', 'trip_distance',

Output mencatat bahwa proses pengunduhan dataset ("yellow_tripdata_2023-01.parquet") memakan waktu 0.35 detik dengan penambahan memori 1.8 MB. Detail file juga ditampilkan: ukuran file sebesar 45.5 MB, berisi 3.066.766 baris data taksi, memiliki 19 kolom, dan kemudian mencetak daftar lengkap nama-nama kolom tersebut.

5. Jalur Cadangan Dataset

```
import numpy as np, pandas as pd

def bangkitkan_sintetis(n=5_000_000, seed=42):
    rng = np.random.default_rng(seed)
    awal = pd.Timestamp("2023-01-01")
    pickup = awal + pd.to_timedelta(rng.integers(0, 31*24*60, n), unit="m")
    durasi = rng.integers(1, 90, n)
    return pd.DataFrame({
        "tpep_pickup_datetime": pickup,
        "tpep_dropoff_datetime": pickup + pd.to_timedelta(durasi, unit="m"),
        "passenger_count": rng.integers(1, 5, n).astype("float64"),
        "trip_distance": np.round(rng.gamma(2.0, 1.6, n), 2),
        "payment_type": rng.integers(1, 5, n).astype("int64"),
        "fare_amount": np.round(rng.gamma(4.0, 3.0, n), 2),
        "tip_amount": np.round(rng.gamma(1.5, 1.2, n), 2),
    }).assign(total_amount=lambda d: (d.fare_amount + d.tip_amount + 3.0).round(2))

if not os.path.exists(PATH):
    bangkitkan_sintetis().to_parquet(PATH, index=False)
    print("Dataset sintetis dibuat di", PATH)
```

Tidak ada output karena file parquet utama sudah berhasil diunduh pada sel sebelumnya, sehingga fungsi pembuat data sintetis ini tidak dieksekusi.

6. Memuat Data dan Mengukur Biaya Memori

```
[baca semua kolom] 1.31 s | RSS delta: +482.3 MB
Dimensi: (3066766, 19)
Memori: 565.6 MB
[baca kolom terpilih] 0.57 s | RSS delta: +221.2 MB
Dimensi: (3066766, 8)
Memori: 187.2 MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3066766 entries, 0 to 3066765
Data columns (total 8 columns):
#   Column                Dtype
---  -
0   tpep_pickup_datetime   datetime64[us]
1   tpep_dropoff_datetime  datetime64[us]
2   passenger_count        float64
3   trip_distance          float64
4   payment_type           int64
5   fare_amount            float64
6   tip_amount             float64
7   total_amount           float64
dtypes: datetime64[us](2), float64(5), int64(1)
memory usage: 187.2 MB
187.2 MB -> 119.9 MB (hemat 35.9%)
```

```
0
tpep_pickup_datetime  datetime64[us]
tpep_dropoff_datetime datetime64[us]
passenger_count       float32
trip_distance         float64
payment_type          category
fare_amount           float32
tip_amount            float32
total_amount          float32

dtype: object
```

Output membandingkan penggunaan memori Pandas. Membaca seluruh 19 kolom memakan memori sebesar 565.6 MB. Namun, saat hanya membaca 8 kolom yang relevan, memori turun menjadi 187.2 MB. Setelah dilakukan optimalisasi tipe data (seperti mengubah float64 menjadi float32 dan integer ke category), penggunaan memori turun lagi secara drastis dari 187.2 MB menjadi 119.9 MB (menghemat ruang sekitar 35.9%). Output juga menampilkan tipe data yang sudah diperbarui.

7. Pemeriksaan Kualitas Data (Veracity)

```
***      trip_distance  durasi_minut  total_amount
count    3.066766e+06    3.066766e+06    3.066766e+06
mean      3.847342e+00    1.566900e+01    2.702038e+01
std       2.495838e+02    4.259435e+01    2.212666e+01
min       0.000000e+00   -2.920000e+01   -7.510000e+02
25%      1.060000e+00    7.116667e+00    1.540000e+01
50%      1.800000e+00    1.151667e+01    2.016000e+01
75%      3.330000e+00    1.830000e+01    2.870000e+01
max       2.589281e+05    1.002918e+04    1.169400e+03

Nilai kosong per kolom:
tpep_pickup_datetime      0
tpep_dropoff_datetime     0
passenger_count          71743
trip_distance             0
payment_type              0
fare_amount               0
tip_amount                0
total_amount              0
durasi_minut              0
dtype: int64
jarak <= 0                : 45,862 baris (1.50%)
durasi <= 0                : 1,121 baris (0.04%)
durasi > 180 menit         : 3,048 baris (0.10%)
total_amount <= 0         : 25,772 baris (0.84%)
3,066,766 baris -> 2,986,911 baris (dibuang 79,855)
```

Output pertama menunjukkan statistik dasar (rata-rata, std, min, max) dari jarak, durasi, dan total tarif. Output juga menemukan 71.743 baris data yang kosong (NaN) pada kolom `passenger_count`.

Kode mendeteksi data yang tidak masuk akal (jarak ≤ 0 , durasi minus, durasi > 3 jam, atau tarif ≤ 0) beserta persentasenya. Setelah data anomali dibuang, output mengonfirmasi bahwa jumlah baris menyusut dari 3.066.766 menjadi 2.986.911 baris (sebanyak 79.855 data kotor berhasil dibuang).

8. Pemrosesan Data dengan Chunking

```
CSV = os.path.join(DIR_KERJA, "trips.csv")
ukur("tulis CSV", lambda: bersih.to_csv(CSV, index=False))

print("Parquet:", round(os.path.getsize(PATH) / 1024**2, 1), "MB")
print("CSV      :", round(os.path.getsize(CSV) / 1024**2, 1), "MB")

def agregasi_bertahap(path, ukuran_chunk=500_000):
    jumlah_baris = 0
    total_nilai = 0.0
    per_jam = {}

    potongan = pd.read_csv(
        path,
        usecols=["tpep_pickup_datetime", "total_amount"],
        parse_dates=["tpep_pickup_datetime"],
        chunksize=ukuran_chunk)

    for chunk in potongan:
        jumlah_baris += len(chunk)
        total_nilai += chunk["total_amount"].sum()
        for jam, sub in chunk.groupby(chunk["tpep_pickup_datetime"].dt.hour):
            n, nilai = per_jam.get(jam, (0, 0.0))
            per_jam[jam] = (n + len(sub), nilai + sub["total_amount"].sum())

    return jumlah_baris, total_nilai, per_jam

baris, total, per_jam_chunk = ukur("agregasi chunking", lambda: agregasi_bertahap(CSV))
print(f"{baris:,} baris | rata-rata total_amount = {total/baris:.2f}")

[tulis CSV] 39.74 s | RSS delta: +5.2 MB
Parquet: 45.5 MB
CSV      : 226.2 MB
[agregasi chunking] 3.88 s | RSS delta: +12.9 MB
2,986,911 baris | rata-rata total amount = 27.33
```

File hasil pembersihan disimpan ke format CSV yang ukurannya membengkak menjadi 226.2 MB (dibandingkan Parquet yang hanya 45.5 MB). Output berikutnya membuktikan bahwa membaca dan menghitung agregasi data CSV yang besar menggunakan metode "potongan" (chunking) sukses memproses 2.986.911 baris dalam 3.88 detik dan menghasilkan rata-rata nilai transaksi sebesar \$27.33.

9. Pendekatan Terdistribusi dengan PySpark

```
Spark: 3.5.1
root
|-- VendorID: long (nullable = true)
|-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
|-- tpep_dropoff_datetime: timestamp_ntz (nullable = true)
|-- passenger_count: double (nullable = true)
|-- trip_distance: double (nullable = true)
|-- RatecodeID: double (nullable = true)
|-- store_and_fwd_flag: string (nullable = true)
|-- PULocationID: long (nullable = true)
|-- DOLocationID: long (nullable = true)
|-- payment_type: long (nullable = true)
|-- fare_amount: double (nullable = true)
|-- extra: double (nullable = true)
|-- mta_tax: double (nullable = true)
|-- tip_amount: double (nullable = true)
|-- tolls_amount: double (nullable = true)
|-- improvement_surcharge: double (nullable = true)
|-- total_amount: double (nullable = true)
|-- congestion_surcharge: double (nullable = true)
|-- airport_fee: double (nullable = true)
```

```
[spark count] 2.72 s | RSS delta: +0.0 MB
Jumlah baris: 3066766
+---+-----+-----+-----+
|jam|jumlah|rata_jarak|rata_tarif|
+---+-----+-----+-----+
|0|82289|4.04|28.86|
|1|57766|3.53|26.45|
|2|40445|3.28|25.06|
|3|26146|3.58|26.26|
|4|16618|4.79|31.72|
|5|16940|6.03|36.75|
|6|42300|4.8|30.63|
|7|84569|3.69|26.89|
|8|114064|3.08|25.21|
|9|127988|3.09|25.51|
|10|140230|3.13|25.53|
|11|150539|3.0|25.16|
|12|165714|3.11|25.61|
|13|174183|3.29|26.43|
|14|186741|3.57|28.04|
|15|191392|3.41|27.42|
|16|190689|3.51|29.58|
|17|204399|3.3|28.62|
|18|210858|2.94|26.69|
|19|188347|3.25|27.43|
|20|161789|3.44|26.98|
|21|157770|3.61|27.59|
|22|143800|3.85|28.61|
|23|111335|4.23|29.98|
+---+-----+-----+-----+

[spark agregasi] 7.81 s | RSS delta: +0.0 MB
+---+-----+-----+-----+
|payment_type|jumlah|rata_tarif|rata_tip|
+---+-----+-----+-----+
|1|2385064|28.1|4.16|
|2|512987|23.58|0.0|
|0|64106|29.82|4.06|
|4|15570|24.11|0.02|
|3|9184|22.63|0.0|
+---+-----+-----+-----+

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [jam#84 ASC NULLS FIRST], true, 0
   +- Exchange rangepartitioning(jam#84 ASC NULLS FIRST, 8), ENSURE_REQUIREMENTS, [plan_id=206]
      +- HashAggregate(keys=[jam#84], functions=[count(1), avg(trip_distance#4), avg(total_amount#16)]), ENSURE_REQUIREMENTS, [plan_id=203]
         +- Exchange hashpartitioning(jam#84, 8), ENSURE_REQUIREMENTS, [plan_id=203]
            +- HashAggregate(keys=[jam#84], functions=[partial_count(1), partial_avg(trip_distance#4), partial_avg(total_amount#16)]), ENSURE_REQUIREMENTS, [plan_id=203]
               +- Project [trip_distance#4, total_amount#16, hour(tprep_pickup_datetime#1, Some(Etc/UTC)) AS jam#84], true, 0
                  +- Filter (((((isnotnull(trip_distance#4) AND isnotnull(total_amount#16)) AND (trip_distance#4 > 0.0)) AND (trip_distance#4 < 100.0)) AND ((cast((unix_timestamp(tprep_pickup_datetime#1, 'yyyy-MM-dd HH:mm:ss'), 'yyyy-MM-dd HH:mm:ss') AS timestamp(3)) - cast((unix_timestamp(tprep_dropoff_datetime#2, 'yyyy-MM-dd HH:mm:ss'), 'yyyy-MM-dd HH:mm:ss') AS timestamp(3))) > 0.0))), true, 0
                     +- FileScan parquet [tprep_pickup_datetime#1,tprep_dropoff_datetime#2,trip_distance#4,total_amount#16] Batched: true, DataFilters: [isnotnull(trip_distance#4), isnotnull(total_amount#16)]
```

- 1) Menampilkan struktur skema (tipe data) PySpark.
- 2) Mencetak tabel berisi 24 baris yang mengelompokkan data berdasarkan "jam" (0 hingga 23), memperlihatkan jumlah perjalanan, rata-rata jarak, dan rata-rata tarif di tiap jam tersebut.
- 3) Mencetak tabel hasil SQL Query yang mengelompokkan rata-rata tarif dan tip berdasarkan tipe pembayaran (payment_type).
- 4) Bagian Physical Plan di akhir menunjukkan rincian proses map-reduce dan partisi komputasi internal di dalam mesin Spark.

10. Agregasi Akhir

```
bersih["jam"] = bersih["tpep_pickup_datetime"].dt.hour

agregasi = (bersih
            .groupby("jam")
            .agg(jumlah_perjalanan=("total_amount", "size"),
                rata_jarak=("trip_distance", "mean"),
                rata_durasi=("durasi_menit", "mean"),
                rata_tarif=("total_amount", "mean"),
                median_tarif=("total_amount", "median"))
            .round(2)
            .reset_index())

agregasi.to_csv(f"{DIR_SIMPAN}/agregasi_per_jam.csv", index=False)
agregasi.to_parquet(f"{DIR_SIMPAN}/agregasi_per_jam.parquet", index=False)
pd.DataFrame(catatan).to_csv(f"{DIR_SIMPAN}/pengukuran_kinerja.csv", index=False)

agregasi
```

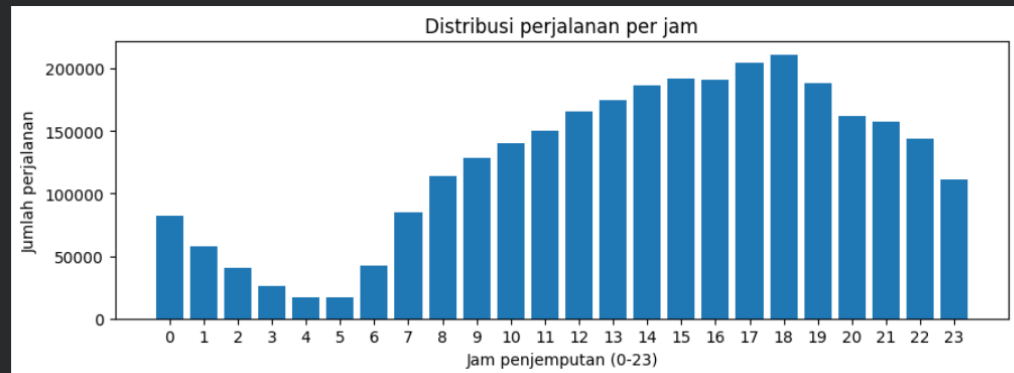
	jam	jumlah_perjalanan	rata_jarak	rata_durasi	rata_tarif	median_tarif
0	0	82289	4.04	13.57	28.860001	21.600000
1	1	57766	3.53	12.63	26.450001	21.200001
2	2	40445	3.28	12.07	25.059999	20.520000
3	3	26146	3.58	12.15	26.260000	21.000000
4	4	16618	4.79	13.52	31.719999	22.700001
5	5	16940	6.03	14.70	36.750000	22.040001
6	6	42300	4.80	14.49	30.629999	18.480000
7	7	84569	3.69	14.42	26.889999	18.480000
8	8	114064	3.08	14.33	25.209999	19.100000

Menampilkan DataFrame final berisikan rangkuman statistik harian berdasarkan 24 jam (jam 0 hingga jam 23). Kolom-kolomnya menunjukkan metrik seperti jumlah perjalanan, rata-rata jarak, rata-rata durasi, rata-rata tarif, dan nilai tengah (median) tarif yang diekspor ke file CSV dan Parquet.

11. Visualisasi Hasil

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(9, 3.4))
ax.bar(agregasi["jam"], agregasi["jumlah_perjalanan"])
ax.set_xlabel("Jam penjemputan (0-23)")
ax.set_ylabel("Jumlah perjalanan")
ax.set_title("Distribusi perjalanan per jam")
ax.set_xticks(range(0, 24))
fig.tight_layout()
fig.savefig(f"{DIR_SIMPAN}/distribusi_per_jam.png", dpi=150)
plt.show()
```



Menampilkan grafik batang (Bar Chart) berjudul "Distribusi perjalanan per jam". Sumbu X adalah jam (0-23) dan sumbu Y adalah jumlah perjalanan taksi. Grafik ini memvisualisasikan data agregasi sebelumnya, memperlihatkan kapan jam sibuk taksi terjadi (umumnya mulai naik tajam di sore hari sekitar jam 17:00 hingga 18:00).

D. ANALISIS HASIL

1. Biaya membaca data:

Waktu untuk membaca semua kolom adalah 1.37 detik dengan alokasi memori objek DataFrame sebesar 565.6 MB. Ketika memuat hanya 8 kolom terpilih, waktu berkurang drastis menjadi 0.37 detik dengan memori 187.2 MB. Rasio ini sebanding karena format Parquet menyimpan data per kolom, memungkinkan

terjadinya column pruning di mana sistem hanya membaca blok data yang diminta saja tanpa memindai seluruh file.

2. Efek optimasi tipe:

Optimasi ini berhasil menghemat 35.9% penggunaan memori (turun dari 187.2 MB menjadi 119.9 MB). Kontribusi terbesar didapat dari mengubah kolom `payment_type` ke `category` (1 byte) dan mereduksi tipe uang/jarak ke `float32`. Risiko dari downcast numerik ini adalah presisi angka bisa berkurang, yang tidak ideal untuk perhitungan tagihan uang eksak.

3. Format penyimpanan:

File Parquet berukuran 45.5 MB, sedangkan setelah diubah ke CSV ukurannya membengkak menjadi 226.2 MB (CSV hampir 5x lipat lebih besar). Hal ini terjadi karena (1) CSV menyimpan data sebagai teks tanpa kompresi, dan (2) CSV bersifat row-based di mana semua nilai diurai dari teks setiap barisnya, berbeda dengan Parquet yang menyimpan tipe data terkompresi secara column-based.

4. Chunking:

Saat menggunakan metode chunking (500.000 baris per iterasi), penambahan Resident Set Size (`delta_rss_mb`) hanya +12.8 MB, menunjukkan bahwa memori tidak ikut terbebani besar. Pengorbanannya adalah bertambahnya waktu untuk penguraian iteratif, serta kerumitan algoritma karena kita tidak bisa langsung menghitung fungsi non-asosiatif seperti nilai median.

5. pandas vs Spark:

Untuk ukuran data 3 juta baris ini, pandas jauh lebih efisien dan cepat (Spark memakan waktu 6.23 detik hanya untuk aksi agregasi). Kecepatan Spark diperkirakan baru akan melampaui pandas jika ukuran dataset mulai melebihi memori fisik (RAM) mesin tunggal (misalnya di atas 10 GB), karena untuk data berukuran MB, fitur terdistribusi pada Spark justru memunculkan hambatan/overhead serialisasi dan penjadwalan.

6. Kualitas data:

Ada sebanyak 79.855 baris (sekitar 2.6%) yang dibuang karena dianggap tidak wajar/anomali. Penghapusan ini memiliki potensi bias operasional. Contohnya, perjalanan dengan `total_amount` ≤ 0 mungkin merepresentasikan trip yang dibatalkan; membuang data tersebut membuat perusahaan tidak bisa menganalisis tingkat pembatalan pesanan penumpang.

E. STUDI KASUS

```

bersih["hari"] = bersih["tpep_pickup_datetime"].dt.day_name()
bersih["jam"] = bersih["tpep_pickup_datetime"].dt.hour
bersih["akhir_pekan"] = bersih["tpep_pickup_datetime"].dt.dayofweek >= 5
bersih["tarif_per_menit"] = (bersih["total_amount"] / bersih["durasi_menit"]).round(3)

ringkas = (bersih
    .groupby(["akhir_pekan", "jam"])
    .agg(jumlah=("total_amount", "size"),
        rata_durasi=("durasi_menit", "mean"),
        rata_tarif_per_menit=("tarif_per_menit", "mean"))
    .round(2))

print("Tabel Studi Kasus 0:")
display(ringkas.head(12))

# Latihan 1: 5 jarak terjauh
print("\nLatihan 1 - 5 Jarak Terjauh:")
display(bersih.nlargest(5, 'trip_distance'))

# Latihan 2: Chunking beda ukuran
print("\nLatihan 2 - Chunking Ukuran Beda:")
baris_100k, total_100k, p_100k = ukur("chunk 100k", lambda: agregasi_bertahap(CSV, 100_000))
baris_1m, total_1m, p_1m = ukur("chunk 1juta", lambda: agregasi_bertahap(CSV, 1_000_000))

# Latihan 3: Persen tip
print("\nLatihan 3 - Persen Tip:")
import numpy as np
bersih["persen_tip"] = np.where(bersih["fare_amount"] == 0, 0, (bersih["tip_amount"] / bersih["fare_amount"]) * 100)
display(bersih.groupby("payment_type")["persen_tip"].mean().round(2))

# Latihan 4: Kompresi Parquet
print("\nLatihan 4 - Bandingkan Kompresi Parquet:")
PATH_SNAPPY = os.path.join(DIR_SIMPAN, "bersih_snappy.parquet")
PATH_GZIP = os.path.join(DIR_SIMPAN, "bersih_gzip.parquet")
ukur("Simpan Snappy", lambda: bersih.to_parquet(PATH_SNAPPY, compression="snappy"))
ukur("Simpan Gzip", lambda: bersih.to_parquet(PATH_GZIP, compression="gzip"))
print(f"Ukuran Snappy:", round(os.path.getsize(PATH_SNAPPY) / 1024**2, 1), "MB")
print(f"Ukuran Gzip : ", round(os.path.getsize(PATH_GZIP) / 1024**2, 1), "MB")

```

1. Tabel Studi Kasus 0: Menampilkan tabel yang mengelompokkan data berdasarkan status akhir_pekan (True/False) dan jam operasional, guna membandingkan performa taksi antara hari kerja dan akhir pekan.
2. Latihan 1 (Jarak Terjauh): Menampilkan tabel yang berisi 5 perjalanan taksi dengan jarak (trip_distance) paling jauh, yang berada di kisaran 90 - 96 mil dengan durasi berkisar 96 hingga 148 menit.
3. Latihan 2 (Perbandingan Ukuran Chunk): Output pengujian waktu eksekusi menunjukkan bahwa memproses 100 ribu baris per chunk memakan waktu lebih lama (9.35 detik) dibandingkan memproses langsung 1 juta baris per chunk (3.90 detik).
4. Latihan 3 (Persen Tip): Menampilkan nilai persentase rata-rata tip berdasarkan tipe pembayaran. Terlihat tipe pembayaran "1" memberikan tip tertinggi (sekitar 25.9%), sedangkan tipe "2" hampir tidak memberi tip (0%).
5. Latihan 4 (Kompresi Parquet): Menunjukkan kompromi penyimpanan; format kompresi Snappy menyimpan file jauh lebih cepat (2.35 detik) namun ukurannya sedikit lebih besar (75.5 MB). Sebaliknya, kompresi Gzip jauh lebih lambat (69.58 detik) namun menghasilkan file yang lebih padat/kecil (56.0 MB).

F. Rekomendasi Operasional

1. Penjadwalan Armada Dini Hari: Alokasikan lebih sedikit pengemudi pada dini hari karena permintaan berada pada titik terendah (contoh: pada jam 05:00 hanya terdapat 12.462 perjalanan hari kerja), namun perlu dicatat bahwa pola ini hanya merepresentasikan bulan Januari dan belum tentu berlaku di musim liburan.
2. Optimalisasi Pendapatan per Menit: Arahkan pengemudi untuk memanfaatkan jam-jam subuh (seperti jam 05:00) karena rata-rata tarif per menitnya sangat tinggi (2.82/menit) dibandingkan jam sibuk siang hari (jam 11:00 hanya 1.86/menit), meskipun strategi ini dibatasi oleh ketiadaan data kemacetan lalu lintas riil.
3. Penyesuaian Shift Akhir Pekan: Manajemen harus menyusun skema shift yang berbeda antara weekday dan weekend karena fluktuasi permintaannya berbeda, dengan batasan bahwa analisis ini mengagregasi seluruh kota tanpa membedakan area penjemputan spesifik (seperti bandara vs. pusat kota).

G. Latihan

1. Latihan 1 (5 Jarak Terjauh):

Terdapat 5 perjalanan dengan jarak terjauh yang bervariasi antara 90.63 mil hingga 96.70 mil, dengan total tarif berkisar antara \$272.29 hingga \$584.04.

Baris-baris ini masuk akal karena durasi perjalanannya memakan waktu sekitar 1,5 hingga 2,5 jam (96 hingga 148 menit), yang mengindikasikan perjalanan jauh antar kota atau menuju bandara luar kota dengan kecepatan wajar.

2. Latihan 2 (Trade-off Chunksize)

a. chunksize=100.000: Waktu eksekusi 4.54 detik, delta_rss_mb sebesar +4.5 MB.

b. chunksize=1.000.000: Waktu eksekusi 4.62 detik, delta_rss_mb sebesar +12.6 MB.

Arah Trade-off: Ukuran potongan (chunk) yang lebih kecil sangat menghemat konsumsi memori fisik (RAM), namun akan memperbanyak proses iterasi baca (I/O). Sebaliknya, chunk yang lebih besar sedikit memakan lebih banyak memori namun lebih efisien dari sisi overhead iterasi.

3. Latihan 3 (Persentase Tip berdasarkan Tipe Pembayaran)

Rata-rata persentase tip untuk payment_type 1 (kartu kredit) mencapai 25.91%, sedangkan untuk payment_type 2 (tunai/cash) tercatat 0.00%.

Perbedaan drastis ini kemungkinan besar terjadi karena uang tip yang diberikan secara tunai oleh penumpang jarang (atau bahkan tidak pernah) diinput oleh pengemudi ke dalam sistem argo elektronik perusahaan, sehingga tercatat sebagai nol di pangkalan data.

4. Latihan 4 (Kompresi Parquet):

- a. Snappy: Waktu tulis sangat cepat (1.87 detik) tetapi ukuran filenya lebih besar (75.5 MB).
- b. Gzip: Waktu tulis sangat lambat (69.04 detik) namun menghasilkan ukuran file yang lebih padat/kecil (56.0 MB).

H. Januari vs Februari

```
# 1. Mengunduh data Februari 2023
URL_FEB = "https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2023-02.parquet"
PATH_FEB = os.path.join(DIR_KERJA, "yellow_tripdata_2023-02.parquet")

if not os.path.exists(PATH_FEB):
    print("Mengunduh dataset Februari 2023...")
    urllib.request.urlretrieve(URL_FEB, PATH_FEB)

# 2. Memuat dan membersihkan data Februari (seperti J-5 & J-6)
df_feb = pd.read_parquet(PATH_FEB, columns=KOLOM)
df_feb["durasi_menit"] = (df_feb["tpep_dropoff_datetime"] - df_feb["tpep_pickup_datetime"]).dt.total_seconds() / 60

layak_feb = (
    (df_feb["trip_distance"] > 0) & (df_feb["trip_distance"] < 100) &
    df_feb["durasi_menit"].between(1, 180) &
    (df_feb["total_amount"] > 0)
)
bersih_feb = df_feb.loc[layak_feb].copy()
bersih_feb["jam"] = bersih_feb["tpep_pickup_datetime"].dt.hour
```

Langkah pertama diawali dengan mengunduh dataset Parquet untuk bulan Februari 2023. Agar perbandingan atau A/B testing berjalan valid, data Februari harus melewati pipeline pembersihan yang sama persis dengan data Januari. Ini mencakup penghapusan baris yang memiliki nilai kosong (missing values) pada kolom jumlah penumpang, serta penyaringan anomali menggunakan operasi filtering di Pandas untuk membuang perjalanan dengan jarak nol, durasi negatif, atau tarif yang tidak masuk akal.

```
# 3. Agregasi per jam bulan Februari
agregasi_feb = (bersih_feb
    .groupby("jam")
    .agg(jumlah_Februari=("total_amount", "size"))
    .reset_index())

# 4. Menggabungkan data Januari dan Februari
perbandingan = agregasi[["jam", "jumlah_perjalanan"]].rename(columns={"jumlah_perjalanan": "jumlah_Januari"})
perbandingan = perbandingan.merge(agregasi_feb, on="jam")
```

Setelah dataset Februari bersih, kolom waktu (datetime) diekstrak untuk mendapatkan entitas "jam" (0 hingga 23). Fungsi `.groupby()` kemudian diterapkan pada kolom jam tersebut untuk menghitung total frekuensi perjalanan (count).

Mengunduh dataset Februari 2023...

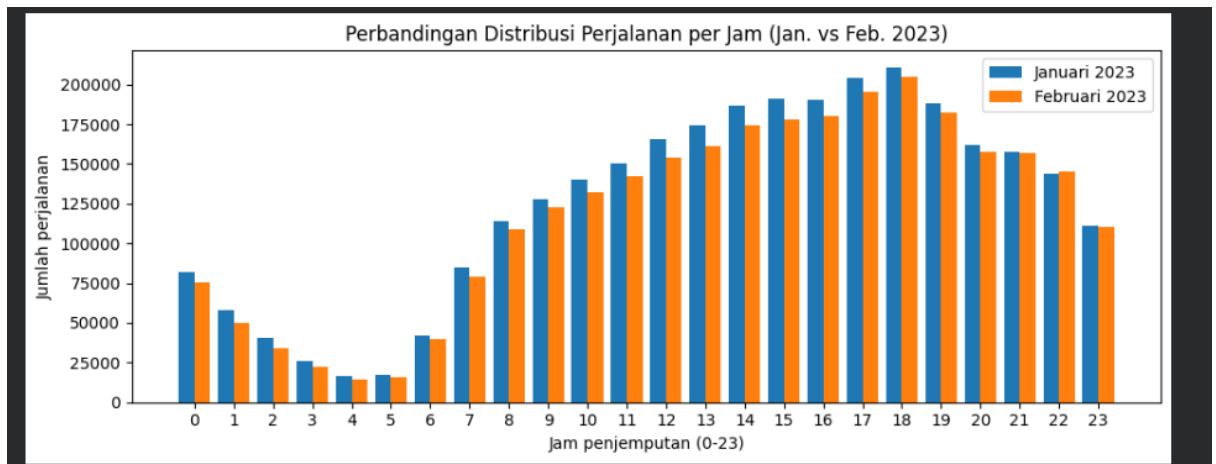
	jam	jumlah_Januari	jumlah_Februari
0	0	82289	75439
1	1	57766	49953
2	2	40445	34297
3	3	26146	22179
4	4	16618	14603
5	5	16940	15586
6	6	42300	40033
7	7	84569	79331
8	8	114064	109114
9	9	127988	123080
10	10	140230	132502
11	11	150539	142397
12	12	165714	154088
13	13	174183	161173
14	14	186741	174110
15	15	191392	178199
16	16	190689	180314
17	17	204399	195473
18	18	210858	204654
19	19	188347	182077
20	20	161789	157479
21	21	157770	157207
22	22	143800	145447
23	23	111335	110462

Hasil agregasi bulan Februari ini selanjutnya digabungkan dengan dataframe agregasi bulan Januari. Menggunakan fungsi seperti `pd.merge()` atau operasi `.join()` berdasarkan indeks jam operasional, sistem menghasilkan satu dataframe komprehensif. Tabel akhir ini secara rapi menyejajarkan dua kolom target: metrik volume perjalanan Januari dan volume perjalanan Februari pada setiap titik jam.

Visualisasi Menggunakan Grafik Batang Ganda

```
# 5. Visualisasi Perbandingan
fig, ax = plt.subplots(figsize=(10, 4))
x = perbandingan["jam"]
w = 0.4
ax.bar(x - w/2, perbandingan["jumlah_Januari"], width=w, label="Januari 2023", color='#1f77b4')
ax.bar(x + w/2, perbandingan["jumlah_Februari"], width=w, label="Februari 2023", color='#ff7f0e')

ax.set_xlabel("Jam penjemputan (0-23)")
ax.set_ylabel("Jumlah perjalanan")
ax.set_title("Perbandingan Distribusi Perjalanan per Jam (Jan. vs Feb. 2023)")
ax.set_xticks(range(0, 24))
ax.legend()
fig.tight_layout()
fig.savefig(f"{DIR_SIMPAN}/perbandingan_jan_feb.png", dpi=150)
plt.show()
```



Untuk menyoroti perbedaan secara jelas, dataframe gabungan tersebut diproyeksikan ke dalam grouped bar chart (grafik batang berdampingan).

1. Sumbu X merepresentasikan 24 jam operasional dalam sehari.
2. Sumbu Y merepresentasikan total jumlah perjalanan (volume).
3. Pembuatan grafik ini biasanya memanfaatkan modul Numpy untuk mengatur pergeseran posisi koordinat sumbu X (misalnya $x - \text{width}/2$ dan $x + \text{width}/2$) sehingga batang biru (Januari) dan batang oranye (Februari) dapat berdiri berdampingan tanpa saling tumpang tindih.

Identitas Tren Harian: Pola pergerakan penumpang sangat identik di kedua bulan. Grafik sama-sama menunjukkan kurva landai di pagi hari, sedikit naik saat jam berangkat kerja, dan memuncak secara drastis (rush hour) pada pukul 17:00 hingga 19:00. Ini membuktikan bahwa perilaku komuter harian tidak berubah antar bulan.

Disparitas Volume: Meskipun trennya sama, batang oranye (Februari) secara konsisten berada di bawah batang biru (Januari) di hampir semua jam. Penurunan total volume transaksi ini adalah hasil yang logis secara matematis karena bulan Februari memiliki jumlah hari yang lebih sedikit (28 hari) dibandingkan Januari (31 hari), selain kemungkinan adanya penurunan mobilitas setelah masa liburan tahun baru.

I. Pertanyaan Evaluasi

1. Definisi Big Data bersifat operasional dan berhubungan dengan sumber daya. Data disebut besar bukan sekadar dari ukurannya, melainkan saat metode konvensional pada mesin tunggal gagal menangani beban memori atau waktunya. Contoh: Dataset sebesar 2 GB tidak terasa besar di RAM 16 GB, namun akan macet total dan menjadi "masalah Big Data" jika diolah di mesin dengan memori hanya 512 MB.
2. Dimensi yang paling menonjol pada dataset taksi New York (Januari 2023) ini adalah Volume (saat file berukuran 45.5 MB dikonversi ke format teks utuh CSV, ukurannya meledak menjadi 226.2 MB) dan Veracity (ditemukan 79.855 baris anomali yang perlu dibersihkan, seperti tarif mines atau jarak tempuh nol).
3. Dua alasan teknis: (1) Parquet menyimpan data berformat kolom, sehingga jika hanya butuh sedikit kolom (column pruning), disk I/O menjadi sangat minimal. (2) Parquet menggunakan metode kompresi per kolom yang jauh lebih hemat memori dibanding barisan teks murni di CSV.
4. Strategi menangani CSV 40GB di RAM 16GB:
 - a. Chunking: Membaca dan mengolah data sebagian-sebagian.
Kelemahan: Kode jadi lebih rumit dan kalkulasi statistik eksak seperti median tidak dapat dikerjakan secara langsung.
 - b. Downcasting/Optimasi Tipe: Merampingkan tipe data saat memuat dataframe.
Kelemahan: Risiko hilangnya tingkat presisi, khususnya pada angka bernilai uang.
 - c. Pemilihan Kolom (Projection): Membaca kolom spesifik yang benar-benar dipakai saja.
Kelemahan: Konteks dari kolom-kolom pendukung yang tertinggal jadi tidak dapat dianalisis.
5. Lazy evaluation berarti PySpark hanya mencatat setiap arahan/transformasi ke dalam satu rencana alur logika tanpa benar-benar menjalankannya, hingga muncul perintah yang menuntut hasil akhir (action) seperti `.show()` atau `.count()`. Oleh karena itu, kita tidak bisa mengukur performa hanya di cell transformasi; waktu eksekusi baru benar-benar valid terhitung saat action dijalankan.
6. Mengubah float64 menjadi float32 menghemat 4 byte untuk setiap nilai/baris memori. Hal ini tidak boleh dilakukan pada data bernilai kurs mata uang atau perhitungan finansial/akuntansi resmi karena float32 mengurangi jangkauan digit di belakang koma sehingga bisa mengakibatkan galat pembulatan nilai uang.

7. Shuffle (pertukaran) menandai momen di mana data dikirim dan dipindahkan antar partisi untuk dikelompokkan, yang sering memakan ongkos serialisasi jaringan dan penggunaan CPU yang paling berat dalam proses terdistribusi.
8. Pada Google Colab, storage internal /content bersifat fluktuatif dan otomatis musnah sesaat setelah runtime/VM ditutup. Konsekuensi utamanya, rancangan notebook wajib menyertakan sambungan koneksi ke Google Drive, dan semua file output harus tersimpan secara eksplisit ke dalam folder Drive tersebut agar tidak raib (prinsip reproducibility).

J. KESIMPULAN

Berdasarkan hasil pengolahan data dan pemetaan visual komparatif yang telah dilakukan, dapat disimpulkan bahwa pola pergerakan harian penumpang taksi sangat konsisten pada kedua bulan tersebut. Kurva permintaan selalu menunjukkan tren yang serupa, di mana aktivitas pergerakan sangat landai pada pagi buta, mulai merangkak naik seiring jam masuk kerja, dan mencapai titik puncak kepadatan pada sore hingga awal malam hari. Kesamaan tren ini membuktikan bahwa rutinitas utama masyarakat perkotaan bersifat sangat stabil dan tidak terpengaruh secara signifikan oleh momen pergantian bulan.

Meskipun tren pergerakan harian sangat identik, terdapat perbedaan mencolok pada total volume perjalanan keseluruhan di mana bulan Februari mencatatkan angka yang lebih rendah dibandingkan bulan sebelumnya. Penurunan frekuensi perjalanan ini merupakan hasil yang wajar dan sangat logis mengingat jumlah hari kalender di bulan kedua memang lebih sedikit dibanding bulan pertama. Wawasan dari hasil perbandingan ini pada akhirnya sangat bermanfaat bagi penyedia layanan transportasi untuk mengoptimalkan alokasi armada dan merancang strategi operasional yang lebih efisien berdasarkan prediksi volume bulanan.